

Short Reflection

Research -> Design -> Build -> Verify

Director of AI

I started by converting the task into constraints and a roleplay instruction, instead of asking questions.

Example prompt

Act as an Azure solution architect. Using only the approved estate in the brief, propose a supportable design for monthly analysis of ~200 NHS organisations

Must include: evidence-linked outputs, human review, monitoring, failure handling, security, cost control and AI Operations handover.

Do not add services unless you can explain why they are required.

Made the business need and non-negotiables explicit.

Kept the model inside Acacium's approved Microsoft Azure.

Used the response as design input, not as an architecture to accept automatically.

Why it helped

Reduced vague suggestions and forced operational concerns into the first design.

What I checked

Every component had to map back to a stated requirement or a clear engineering need.

Human decision

I selected the final boundaries and how they were represented in the diagram.

Use of Plugins & MCPs, Context7

Back-end way to retrieve only the relevant open source documentation.

Example prompt

Before recommending Azure components, use Context7 via MCP to retrieve the current official documentation for only the decision I'm making

Return:

1. the relevant capability
2. limitations
3. security or identity considerations
4. the exact point that changes my architecture decision

Define a clear scope with limitations, e.g. keep unrelated documentation out of the AI's context space.

Reduced token use by retrieving open source documentation slices as markdown.

Saved time by not needing to search the Internet in order to web scrape data before interpreting the open source documentation.

Avoided relying on stale model memory for changing platform details.

Best use

Service limits, SDK behaviour, identity, observability and integration details.

Avoided

Copying large documentation pages into the conversation or trusting generic summaries.

Judgement

I still checked whether the documented feature actually solved the business requirement.

Software Engineer, Software Architect

In order to adapt, Software Engineers of today must become Software Architects.

Example prompt

Review this proposed architecture from three perspectives:

- AI Operations: What will be difficult to run every month?
- Governance (Risk Management & Security): Where could data, identity or prompt controls fail?
- BD user: What would stop a non technical user trusting the output?

Critique by deliberately asking the hardest questions first.
Then propose only the changes of which that actively improve design.

Pattern

Initial design -> challenge -> targeted refinement.

Benefit

Found omissions without expanding the architecture for its own sake.

Decision rule

Only keep a change when it improves supportability, control or user trust.

Deep Iterative Technical Plan

The only useful prompt loop is to treat the implementation plan as something to review, score and improve.

Example prompt

Score this implementation plan out of 100 against the latest and greatest software engineering best practices of 2026 for this tech stack

Explain the specific gaps preventing 100/100, prioritised by risk.

Then rewrite the plan to close those gaps. This includes: interfaces, failure paths, security, observability, test coverage, rollback and handover.

Plan v1 -> Critique -> Gap list -> Plan v2

...

Never blindly accept the first plan.

Why score it?

The number was less important than forcing explicit gap analysis.

What improved

Dependencies, edge cases, test strategy and operational concerns became implementation tasks.

Control

The plan became the contract the coding agent was expected to follow.

Goal Setting

Once the plan is set, define a clear goal.

Example prompt

Implement the approved plan against the existing repository...

Before editing:

- inspect the relevant files and conventions
- preserve existing interfaces unless the plan requires a change
- identify the smallest set of files to modify

During implementation:

- make one coherent change at a time
- add tests alongside behaviour
- stop and surface a conflict rather than inventing a workaround

Reduced broad, speculative code generation.

Made repository context part of the task instead of relying on assumptions.

Prompt structure

Role, context, constraints, explicit output.

Agent behaviour

Inspect first, implement second, surface uncertainty.

My role

Approve trade offs and keep the implementation aligned with the design.

~~Copy and Paste Prompts~~, Programmatic Guardrails

Enforce coding standards yourself, for less prompt overhead.

Example prompt

Do not mark the implementation complete until the relevant quality gates pass

Run and fix iteratively:

- pre-commit and pre-push hooks
- formatting, linting, static analysis and type checks
- unit and integration tests
- make qa
- make test-integration
- pip-audit
- just qa
- just bootstrap

Never use --no-verify or any hook bypass.

Code
|
Static QA
|
Tests
|
Audit
|
Ready

Kept recurring rules consistent across iterations.

Reduced re-explaining the same expectations for every single prompt.

Principle

Confidence comes from standardisation, not from quick generated code

Prevent failure loop

Run -> inspect error -> make smallest fix -> rerun.

Maintainability

Guardrails can be reviewed and changed independently of feature prompts.

Front-end Testing

Browser and computer use for testing.

Example prompt

Start the local application and verify the implemented user journey end to end

For each acceptance criterion:

1. perform the action in the browser
2. inspect the visible result
3. Inspect traceback logs
4. capture evidence
5. fix any mismatch and repeat

Do not infer success from the code diff alone.

Caught integration and UX issues that unit tests may miss.

Linked front-end behaviour to back-end execution evidence.

Made the final review reproducible rather than based on a narrative.

Front end

Browser verification against the actual user flow.

Back end

CLI / service logs used to confirm processing and failures.

Evidence

Screenshots and logs provide a stronger completion signal than explanation.